



UNIVERSITÀ DI PARMA

Bitwise Operators

A Bit of Fun

- Bitwise operators
- Reason why
- Single & multiple bits operations
- Shift operators

SUMMARY



- Memory is composed by “bits”
- Every data is then stored in memory in binary
- Usually we do not need to access the single bit
- C anyway provides us specific operators that work at bit level

- Why we should deal with single bits?
 - Access to raw HW often requires to deal with single bits
 - More efficient arithmetic operations
 - More efficient storing of data

- They can work on single bits

– $\&$ → binary AND 

– $|$ → binary OR 

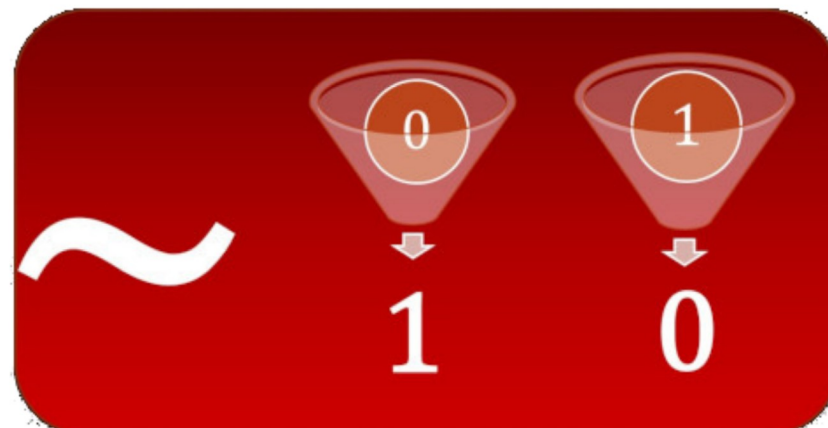
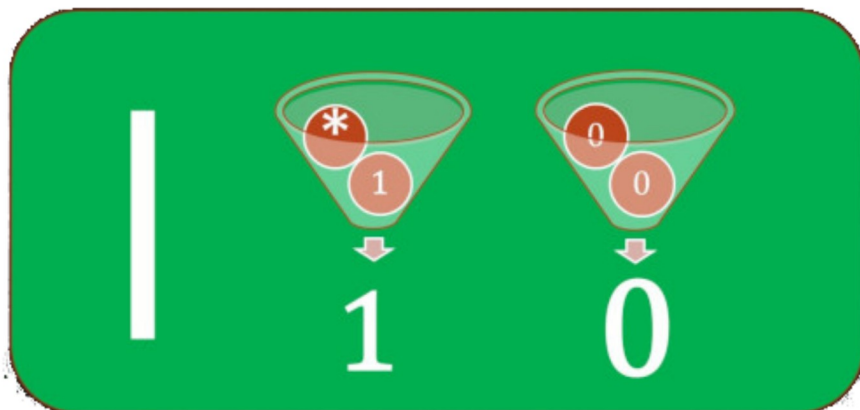
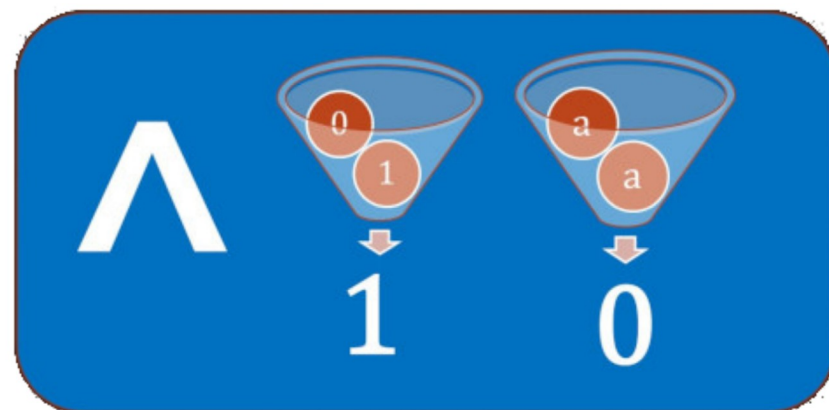
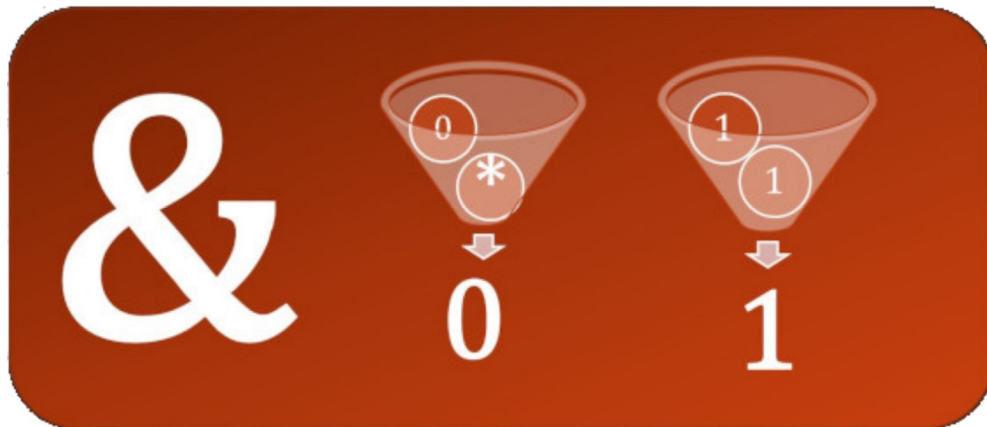
– $\sim$ → binary NOT 

– $\wedge$ → binary XOR 

– $<<$ → left shift

– $>>$ → right shift

Single bit operations



Multiple bits operations

- These operators are applied to integer types
 - Usually multiple bits!
- The operator is applied to the corresponding bits in each data

AND	
	0110
&	1100
	0100

OR	
	0110
	1100
	1110

XOR	
	0110
^	1100
	1010

NOT	
~	1100
	0011

- Note that results are different from the same logical operator!

- Shift all the bits of a data to the right or to the left
- Look at the following example
- We want to shift 213 by 3

213

1	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

213 << 3 1 1 0

1	0	1	0	1	?	?	?
---	---	---	---	---	---	---	---

213 >> 3

?	?	?	1	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---

- In both cases
 - Bits shifted off the end are lost

213

1	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

213 << 3 1 1 0

1	0	1	0	1	?	?	?
---	---	---	---	---	---	---	---

213 >> 3

?	?	?	1	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---

- Not symmetrical behavior
 - Left shift $\rightarrow$ new lower order bits are filled in with 0s
 - Right shift $\rightarrow$ highest order bit is replicated

213

1	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

213 << 3

1	0	1	0	1	0	0	0
---	---	---	---	---	---	---	---

213 >> 3

1	1	1	1	1	0	1	0
---	---	---	---	---	---	---	---

- $|$ with 1 is useful for turning select bits on
- $\&$ with 0 is useful for turning select bits off
- $|$ is useful for taking the union of bits
- $\&$ is useful for taking the intersection of bits
- $\wedge$ is useful for flipping select bits
- $\sim$ is useful for flipping all bits

- Integer number manipulation
 - Get/extract a specific byte of an integer or specific bits
 - e.g., complement letter case
- Bit masking for flags/configuration etc.
- Fast math
 - Left/right shifts are multiplication/divide by two
 - Trick for power of 2 check



UNIVERSITÀ DI PARMA

Le Espressioni



KEEP
CALM
IT'S
QUESTION
TIME

We must either institute conventional forms of expression or else pretend that we have nothing to express.

George Santayana, Soliloquies in England